

4. Data Models

4a. RDS PostgreSQL

Relational data with moderate write traffic. Used as the single source of truth for structure and membership — avoiding dual-write complexity.

channels

Column	Type	Notes
id	UUID PK	
name	VARCHAR	
org_id	UUID	
created_at	TIMESTAMPTZ	

photos

Column	Type	Notes
id	UUID PK	
channel_id	UUID FK	→ channels
uploader_id	UUID FK	→ users (external)
s3_key	VARCHAR	
status	ENUM	PENDING READY
last_activity_at	TIMESTAMPTZ	used for gallery sort
created_at	TIMESTAMPTZ	

annotations

Column	Type	Notes
id	UUID PK	
photo_id	UUID FK	→ photos
created_by	UUID FK	→ users
cx	FLOAT	circle center x, % of image width
cy	FLOAT	circle center y, % of image height
r	FLOAT	radius, % of image width
status	ENUM	OPEN RESOLVED
created_at	TIMESTAMPTZ	

Geometry stored as percentages so circles render correctly regardless of device screen size or zoom level.

thread_members

Column	Type	Notes
annotation_id	UUID PK	composite PK
user_id	UUID PK	composite PK
added_at	TIMESTAMPTZ	upserted on each @mention

4b. DynamoDB — messages

High-write, cursor-paginated, eventually consistent. No joins required.

Attribute	Type	Notes
PK	String	annotation_id
SK	String	created_at#message_id (composite, ensures uniqueness + order)
user_id	String	
body	String	max 400 KB per item
mentions	StringSet	user IDs tagged with @mention
created_at	ISO8601	

Access patterns

- **Load thread:** Query PK=annotation_id, ScanIndexForward=true, Limit=20, paginate via LastEvaluatedKey
- **Post message:** PutItem with PK + composite SK
- **Notification fan-out:** REST API reads thread_members from RDS — no GSI needed